

Detailed Implementation Plan for a Hybrid Exasim Architecture

1. Summary

Yes. The maintainable implementation strategy is to **shrink the template boundary to the model/kernel edge** and move everything else into a compiled core.

A concrete plan for Exasim is:

- keep `MyModel` and the model-dependent Kokkos kernels templated
- replace `CDiscretization<M>`, `CSolution<M>`, `CSolver<M>`, `CPreconditioner<M>` with non-templated core classes
- introduce a small host-side `ModelOps` / `ModelContext` adapter that launches model-specialized kernels
- keep `ModelContext` as a simple struct, not a heavy class
- pass `ModelContext` into the `CSolution(...)` and `CDiscretization(...)` constructors so constructor-time initialization can use model-dependent init operations
- compile the backend into `libexasim_core.a`
- keep `run<M>` and `ExasimSolver<M>` as thin templated frontends over the non-templated core
- isolate the old ABI path behind one compatibility adapter instead of keeping `AbiAdapter` in the main architecture

2. Assumptions

- The current template spread is the one visible in:
 - `backend/Discretization/discretization.h`
 - `backend/Solution/solution.h`
 - `backend/Solver/solver.h`
 - `backend/Preconditioning/preconditioner.h`
- The current public package is the header-only `Exasim::headers` target in `install/CMakeLists.txt`.
- Model-dependent launches already have a natural boundary in:
 - `include/exasim/drivers.hpp`
 - `include/exasim/kernels/`

- `readsolstruct` in `backend/Discretization/readbinaryfiles.hpp` computes initial fields through `cpuInit*Driver` calls, so if `cpuInit*` is moved into `ModelOps`, `ModelContext` must be available during construction.
- Host-side dispatch through function pointers is acceptable if it happens at driver-launch granularity, not inside inner device loops.

3. Design reasoning

The current `library-port` over-templates the backend.

The symptom is that `M` leaks into:

- discretization
- solution
- solver
- preconditioner
- point locator
- output/QoI paths
- install/public headers

That causes:

- template contagion
- backend `.hpp` growth
- `.cpp` files being included into headers
- `inline/ODR` workarounds like `backend/Model/KokkosDrivers.cpp`
- a very large public transitive include graph

The preferred lifecycle is also important:

- `ModelContext` is created at the templated API boundary
- `CSolution(...)` receives `ModelContext` at construction time
- `CSolution(...)` passes `ModelContext` into `CDiscretization(...)` at construction time
- constructor-time paths such as `readsolstruct` can immediately use `getModelContext()`
- `bindModelContext(...)` is no longer the primary initialization path; at most it is a secondary/rebind mechanism

The better boundary is:

- **templated:** only code that truly needs `M::flux`, `M::source`, `M::fbou_hdg`, Jacobians, and compile-time sizes
- **non-templated:** orchestration, ownership, file I/O, solver flow, mesh/preprocessing, point location, output plumbing

That means the current `EXASIM_DRIVER_CALL(...)` model should be replaced by a runtime `ModelOps` object whose function pointers point to **host-side wrappers** specialized on `M`. Those wrappers then launch the templated kernels.

4. Proposed implementation

Phase 0: Freeze invariants and scope

Affected files:

- `docs/library_port_implementation_strategy.tex`
- add a new architecture note under `docs/` or `docs/03-internals/`

Plan:

- Define the non-negotiable numerical and API invariants before refactoring.
- Lock down that HDG assembly structure must remain unchanged.
- Write a one-page boundary note: “templated kernel edge / compiled runtime core.”
- Identify which public entry points must remain source-compatible:

- `exasim::run<M>(...)`
- `ExasimSolver<M>`
- `legacy backend/Main/main.cpp`

Deliverable:

- design note with exact boundary and migration rules

Phase 1: Introduce a runtime model interface

New files:

- `include/exasim/model_ops.hpp`
- `include/exasim/model_context.hpp`
- `include/exasim/model_adapter.hpp`
- `backend/Model/model_ops.cpp` only if shared non-template helpers are needed

Plan:

- Define a `ModelOps` struct of host-callable function pointers for each driver family currently reached through `EXASIM_DRIVER_CALL(...)`.

- Split the ops by category:
 - volume: `flux`, `source`, `sourcew`, `eos`, `tdfunc`
 - boundary/interface: `fbou`, `fhat`, `fint`, `fext`, `ubou`
 - init: `initu`, `initudg`, `initwdg`, `initodg`
 - output/QoI/vis: `output`, `monitor`, `qoi_volume`, `qoi_boundary`, `vis_scalars`, `vis_vectors`, `vis_tensors`
- Define `ModelContext` as:
 - `const ModelOps* ops`
 - model metadata: `nd`, `ncu`, `ncw`, `nco`, `disc`, maybe feature flags
 - optional opaque pointer for model-owned state, if ever needed later
- Keep `ModelContext` as a plain struct used only to carry dispatch pointers and model metadata. Do not make it a heavy class unless it later needs ownership or invariant logic.
- Build `ModelContext` at the templated boundary only, then pass it into `CSolution(...)` and `CDiscretization(...)` as part of normal construction.
- `cpuInitu`, `cpuInitudg`, `cpuInitwdg`, and `cpuInitodg` must be represented in `ModelOps`, because initialization is model-dependent and may run during `readsolstruct`.

Important implementation detail:

- These function pointers must point to host functions that launch Kokkos kernels specialized on `M`.
- Do not use runtime polymorphism inside device code.

Deliverable:

- a runtime model interface that can replace the macro-based dual dispatch

Phase 2: Build templated model adapters

New files:

- `include/exasim/model_adapter_impl.hpp`
- `backend/ModelAdapters/abi_adapter.cpp`
- generated or handwritten adapter TUs, for example:
 - `apps/library_example/poisson2d/model_adapter.cpp`
 - generated codegen output `.../my_model_adapter.cpp`

Plan:

- Implement `template <class M> ModelOps make_model_ops();`
- Each function in `ModelOps` forwards to one current templated driver in `include/exasim/drivers.hpp`.

- Example pattern:
 - `ops.flux = &flux_driver_adapter<M>;`
 - where `flux_driver_adapter<M>(...)` calls `exasim::FluxDriver<M>(...)`
- This is the key move that narrows the template boundary without losing specialization.

Deliverable:

- one model adapter TU per model family or executable
- no need for large backend classes to depend on M

Phase 3: Replace templated backend classes with core classes

Affected files:

- `backend/Discretization/discretization.h`
- `backend/Discretization/discretization.hpp`
- `backend/Solution/solution.h`
- `backend/Solution/solution.hpp`
- `backend/Solver/solver.h`
- `backend/Solver/solver.hpp`
- `backend/Preconditioning/preconditioner.h`
- `backend/Preconditioning/preconditioner.hpp`
- `backend/PointLocator/pointlocator.h`
- `backend/PointLocator/pointlocator.hpp`

Plan:

- Introduce non-templated replacements:
 - `CDiscretization`
 - `CSolution`
 - `CSolver`
 - `CPreconditioner`
- Replace `CDiscretization<M>&` references with `CDiscretization&`.
- Store `const ModelContext*` or `const ModelOps*` inside `CDiscretization`, and initialize it during construction.
- Update constructors so `ModelContext` is part of the construction dependency graph:
 - `CDiscretization(const ModelContext&, ...)`
 - `CSolution(const ModelContext&, ...)`

- `CSolver(...)` and `CPreconditioner(...)` receive it directly only if they need constructor-time model access
- Keep optional bind methods only as secondary/rebind hooks:
 - `CDiscretization::bindModelContext(const ModelContext&)`
 - `CSolver::bindModelContext(const ModelContext&)` if solver-side access is needed
 - `CPreconditioner::bindModelContext(const ModelContext&)` if preconditioner-side access is needed
 - `CSolution::bindModelContext(const ModelContext&)` as the top-level coordinator for rare rebind flows
- Keep the constructor flow centered on `CSolution(...)`. The constructor now receives `ModelContext` and propagates it to `disc` immediately so constructor-time paths such as `readsolstruct` can use model-dependent init operations.
- If retained, `CSolution::bindModelContext(...)` is not the main public initialization path. It is only for controlled rebind or reconstruction scenarios.
- Add accessor functions instead of exposing raw members:
 - `CDiscretization::getModelContext()`
 - `CSolution::getModelContext()` if the solution object needs to expose the bound context
- Use `getModelContext()` as the checked accessor name rather than `model_context()` or `ModelContext()` to avoid confusion with the `ModelContext` type itself.
- Add an internal validity check on bind:
 - `disc.common.nd == model.nd`
 - `disc.common.ncu == model.ncu`
 - `disc.common.ncw == model.ncw`
 - discretization kind is compatible with `model.disc`
- Convert all current template-heavy helper signatures in backend `.hpp` files to operate on the core types plus `ModelContext`.

Deliverable:

- non-templated backend class graph
- template parameter removed from the core ownership and orchestration classes
- constructor-time `ModelContext` availability for all initialization paths that need model-dependent operations
- checked access through `getModelContext()` rather than raw direct access

Phase 4: Eliminate EXASIM_DRIVER_CALL(...) from the main backend

Affected files:

- include/exasim/detail/driver_dispatch.hpp
- all backend call sites using EXASIM_DRIVER_CALL, especially:
 - backend/Discretization/residual.hpp
 - backend/Discretization/uequation.hpp
 - backend/Discretization/uresidual.hpp
 - backend/Discretization/qaicalculation.hpp
 - backend/Discretization/getuhat.hpp
 - backend/Discretization/discretization.hpp
 - backend/Discretization/postdiscretization.hpp

Plan:

- Replace macro dispatch with explicit runtime calls:
 - `disc.getModelContext().ops->flux(...)`
 - `disc.getModelContext().ops->source(...)`
 - etc.
- Keep the old macro only inside the compatibility layer during migration.
- Make backend residual/assembly code ordinary non-template code unless the algorithm truly depends on compile-time dimensions.
- Route constructor-time initialization through the same runtime dispatch system, for example `getModelContext().ops->initu(...)` and related init operations instead of direct legacy-only `CpuInit*Driver` calls.

Deliverable:

- macro removed from the main backend path
- much smaller template footprint in `backend/Discretization/*.hpp`

Phase 5: Remove .cpp inclusion from headers and build a static core

Affected files:

- backend/Discretization/discretization.hpp
- backend/Discretization/postdiscretization.hpp
- backend/Model/KokkosDrivers.cpp
- backend/Model/ModelDrivers.cpp
- install/CMakeLists.txt

Plan:

- Stop `#include-ing` `backend/Model/KokkosDrivers.cpp` and similar `.cpp` files from backend headers.
- Build a real static target, for example:
 - `add_library(exasim_core STATIC ...)`
 - export as `Exasim::core`
- Move non-template implementations into normal `.cpp` translation units.
- At this point, `inline` on those free functions should no longer be necessary.

Deliverable:

- `libexasim_core.a`
- no more “included `.cpp` as implementation header” pattern in the core

Phase 6: Refactor public API into thin templated wrappers**Affected files:**

- `include/exasim/run.hpp`
- `include/exasim/solver_facade.hpp`
- public wrappers under `include/exasim/`

Plan:

- Keep `run<M>` templated, but make it thin:
 - build `ModelContext ctx = make_model_context<M>();`
 - construct `CSolution(ctx, ...)` through the usual file/runtime path
 - then call non-templated `run_impl(...)`;
- Keep `ExasimSolver<M>` templated only at construction/binding points:
 - `ExasimSolver<M>` owns `std::unique_ptr<CSolution>`
 - it constructs the solution object with a ready `ModelContext`
- Remove backend-heavy includes from public headers where possible.
- Make public headers depend on stable declarations, not large backend internals.

Deliverable:

- same user-facing syntax
- much thinner template layer

Phase 7: Isolate the legacy ABI path

Affected files:

- `include/exasim/detail/abi_adapter.hpp`
- `backend/Main/main.cpp`
- `backend/Model/libpdemodel.hpp`
- `backend/Model/libpdemodel.cpp`
- `legacy ModelDrivers.cpp` / `KokkosDrivers.cpp`

Plan:

- Replace “ABI is one branch of the main architecture” with “ABI is one adapter implementation of `ModelOps`.”
- Create `make_model_ops_abi()` that binds the old driver symbols.
- Keep `backend/Main/main.cpp` using that adapter.
- Remove ABI branching from the main template/runtime design.

Deliverable:

- ABI path survives
- but no longer shapes the public or core architecture

Phase 8: Packaging and install model

Affected files:

- `install/CMakeLists.txt`
- `tests/install_consumer/CMakeLists.txt`
- `tests/multi_tu_consumer/`

Plan:

- Replace header-only export with:
 - `Exasim::core` static library
 - optional small `Exasim::model_api` interface target if needed
- Install only the public API headers and genuinely templated kernel headers.
- Stop installing large parts of `backend/` as public transitive implementation detail.
- Keep consumer tests:
 - out-of-tree install test
 - multi-TU consumer test

- one handwritten model consumer
- one generated model consumer

Deliverable:

- cleaner install tree
- smaller public header surface
- simpler consumer story

Phase 9: Controlled migration sequence

Order of execution:

1. add `ModelOps` and `ModelContext`
2. add templated adapters for current kernels
3. introduce core classes in parallel with current templated classes
4. migrate one path end-to-end:
 - suggested pilot: handwritten `poisson2d`
5. build `Exasim::core`
6. convert `run<M>` and `ExasimSolver<M>`
7. migrate ABI path behind adapter
8. delete old wide-template path after equivalence is proven

This sequencing matters. Do not flip the whole repo at once.

5. Risks

- The biggest technical risk is placing too much behind runtime dispatch and losing compile-time specialization where HDG kernels need it.
- The second risk is trying to make the core non-templated too quickly; some helper layers in `backend/Discretization/` and `backend/Solver/` may still carry hidden compile-time assumptions.
- The migration can temporarily increase complexity if old and new paths coexist for too long.
- Constructor-time `ModelContext` means the core constructors depend on the model-dispatch setup earlier than before; this is correct for Exasim if initialization uses `cpuInit*` through `ModelOps`, but it raises the bar for constructor correctness.
- GPU builds are the most sensitive phase because kernel launch wrappers, device compilation, and linkage all change at once.
- The current branch also has local worktree changes, so any implementation should start from a clean branch.

6. Verification plan

For each phase, require these checks.

Numerical equivalence

- same residual/Jacobian behavior for one HDG scalar case
- same QoI and output for one steady case
- same time-dependent behavior for one DIRK case

Build/link correctness

- single-TU consumer
- multi-TU consumer
- install-tree `find_package(Exasim)` consumer
- no `.cpp` implementation includes in `public/core` headers after Phase 5
- verify that constructor-time `readsolstruct` can compute initial fields through `ModelOps` init functions

Backend coverage

- CPU single-rank
- CPU MPI
- NVIDIA GPU
- one ABI-legacy executable still working through the compatibility adapter

Architecture acceptance criteria

- `backend/Main/main.cpp` no longer determines the main design
- `AbiAdapter` no longer appears in core class templates
- `KokkosDrivers.cpp` is compiled, not included
- `run<M>` and `ExasimSolver<M>` are thin wrappers, not owners of the backend architecture
- `install/CMakeLists.txt` exports `Exasim::core`, not only `Exasim::headers`
- `CSolution(...)` and `CDiscretization(...)` receive `ModelContext` at construction time
- `getModelContext()` is the standard checked accessor for bound model dispatch

If you want, I can turn this into a **work-package plan with estimated effort, file ownership, and merge order**, or into a **refactor RFC document** under `docs/`.